Name: Praju Maharjan

Group: L5CG9

Workshop - 3

1. Write a C program that prompts the user to enter a word and then prints the entered word. Modify the program by using pointers to check whether the entered word is a palindrome.

Ans :

Source Code :

```
#include<stdio.h>
#include <string.h>

void check_palindrome(char* word){
 int size=strlen(word)-1;
 char reversed_word[size];

 for(int i=0;i<=size;i++){
  reversed_word[i]=word[size-i];
 }
 if(!(strcmp(word,reversed_word))){
  printf("%s is a palindrome.",word);
 }else{
  printf("%s is not a palindrome.",word);
 }
}

int main(){
 char word[50];
 printf("Enter a word : ");
 scanf("%s",word);
 printf("The word you entered is : %s.\n",word);

 char *pointer=word;

 check_palindrome(pointer);

 printf("\n");
 return 0;
}
```

Output :

```
Enter a word : hello
The word you entered is : hello.
hello is not a palindrome.
```

```
Enter a word : noon
The word you entered is : noon.
noon is a palindrome.
```

2. Write a C program that asks the user to enter a number between 1000 and 99999, dynamically allocates an integer array of that size using malloc, fills the array with random numbers using the time.h library, checks each number to determine if it is a prime number, prints all the prime numbers found, and finally frees the allocated memory.

Ans :

Source Code :

```c
#include <stdio.h>
#include <time.h>
#include<stdlib.h>

int isPrime(int number){
 if(number<=1){
  return 0;
 }
 if(number==2){
  return 1;
 }

 for(int i=2;i*i<=number;i++){
  if(number%i==0){
   return 0;
  }
 }
 return 1;
}

int main(){
 int *ptr,n;
 srand(time(NULL));
 while(1){
  printf("Enter a number between 1000 and 99999 : ");
  scanf("%d",&n);
  if(n<1000 || n>99999){
   printf("Please enter a number between 1000 and 99999.\n");
   continue;
  }
  ptr=(int*) malloc(n*sizeof(int));

  for(int i=0;i<n;i++){
   *(ptr+i)=(rand() % 1000);
  }
  printf("The prime numbers in the array(numbers repeated) are : \n");
  for(int i=0;i<n;i++){
   if(isPrime(*(ptr+i))){
    printf("%d\n",*(ptr+i));
   }
  }

  free(ptr);
  break;
 }
 printf("\n");
 return 0;

}
```

Output :

```
Enter a number between 1000 and 99999 : 2
Please enter a number between 1000 and 99999.
Enter a number between 1000 and 99999 : 1000
The prime numbers in the array(numbers repeated) are :
239
479
317
643
677
823
281
389
577
307
59
157
727
829
947
223
683
31
41
317
157
127
811
23
541
563
719
823
653
761
263
229
739
269
269
577
197
797
839
421
971
7
251
```

All printed numbers not included

3. Write a C program to implement the “selection sort” algorithm to sort an integer array.

Ans :

Source Code :

```
#include<stdio.h>

void selection_sort(int arr[]){
 for(int i=0;i<10;i++){
  int index_of_smallest_number=i;

  for(int j=i+1;j<10;j++){
   if(arr[j]<arr[index_of_smallest_number]){
    index_of_smallest_number=j;
   }
   int temp=arr[index_of_smallest_number];
   arr[index_of_smallest_number]=arr[i];
   arr[i]=temp;
  }
 }
 printf("\nThe sorted array : \n");
 for(int i=0;i<10;i++){
 printf("%d ",arr[i]);
 }

}

int main(){
 int arr[10]={7,5,3,4,8,1,6,10,9,2};

 printf("The unsorted array : \n");
 for(int i=0;i<10;i++){
 printf("%d ",arr[i]);
 }

 selection_sort(arr);

 printf("\n");
 return 0;
}
```

Output :

```
The unsorted array :
7 5 3 4 8 1 6 10 9 2
The sorted array :
1 2 3 4 5 6 7 8 9 10
```

4. Write a C program to implement the “Bubble sort” algorithm to sort a string into alphabetical order.

Ans :

Source Code :

```
#include<stdio.h>

void bubble_sort(int arr[]){
 int temp;
 for(int i=0;i<10;i++){
  for(int j=0;j<=(10-i-1);j++){
   if(arr[j]>arr[j+1]){
    temp=arr[j];
    arr[j]=arr[j+1];
    arr[j+1]=temp;
   }
  }
 }

 printf("\nThe sorted array : \n");
 for(int i=0;i<10;i++){
 printf("%d ",arr[i]);
 }

}

int main(){
 int arr[10]={7,5,3,4,8,1,6,10,9,2};

 printf("The unsorted array : \n");
 for(int i=0;i<10;i++){
 printf("%d ",arr[i]);
 }

 bubble_sort(arr);

 printf("\n");
 return 0;
}
```

Output :

```
The unsorted array :
7 5 3 4 8 1 6 10 9 2
The sorted array :
1 2 3 4 5 6 7 8 9 10
```